

COURSE 3 - DATA ENGINEERING 2026**C3-103 - TransitOps Production Drill**

Integrated independent production drill on a fresh shipment domain. Limited hints only after a genuine attempt.

Artifact	C3-103
Status	2026 - LEARNER EDITION
Phase	C3-09 Production Capstone & Engineering Transition 15 lessons 36-48 focused hours
Prerequisite	C3-08 Gate PASS; all earlier C3 evidence retained.
Assessment boundary	C3-104 is the phase Gate only. C3-113 remains the later course-wide Final Production Gate.
Brand	Charcoal #1F2937 Teal #14B8A6 Soft Blue #3B82F6 AMOLED #000

House teaching rule: requirement -> mental picture -> small proof -> guided integrated build -> expected result -> why it worked -> break/fix -> changed independent case -> validation -> production reasoning -> evidence. Integration is not mastery until the learner can explain and recover the system independently.

Scenario - TransitOps

TransitOps receives shipment records and publishes trustworthy route-performance data. Build a coherent batch/current-state path, apply the correction, define distributed scale transfer, operate one controlled failure, and hand the system to another engineer.

Required work

- State shipment_id grain, source contract and correction semantics.
- Build batch -> current state -> route metric path; prove T903 correction exactly once under replay.
- Define Bronze/Silver/serving contracts and bounded orchestration/backfill behavior.
- Include a Spark component or explicit executable scale-transfer path with parity invariant.
- Block unsafe publish with quality/reconciliation controls.
- Define freshness + validated-publish SLOs and correlated run evidence.
- Define least-privilege and sanitized-evidence policy.
- Inject a different invalid delivered-duration defect in a disposable copy; contain, repair, rerun/backfill and reconcile.
- Create CI/deployment checklist, runbook and concise handoff.
- Defend why streaming is included or excluded from the business latency requirement.

Completion contract

PASS requires every required competency below to PASS and zero unresolved critical failures. A local design can prove local competencies; live Airflow/Spark/Fabric/GitHub-OIDC claims require matching runtime evidence.

Required competencies

- Requirements/contracts
- Batch + correction/replay correctness
- Layered architecture/serving
- Orchestration/reprocessing
- Distributed scale-transfer parity
- Quality/reconciliation
- Observability/SLO/recovery
- Security/change safety
- Incident response
- Reproducibility/handoff/defense

Critical failures

- Fabricated heavy-tool/cloud execution evidence
- Secret/PII exposure
- Wrong grain/duplicate current state after replay
- Unsafe recovery or unbounded destructive rebuild
- Quality failure allowed to publish
- No post-recovery reconciliation
- Opening protected review before genuine attempt